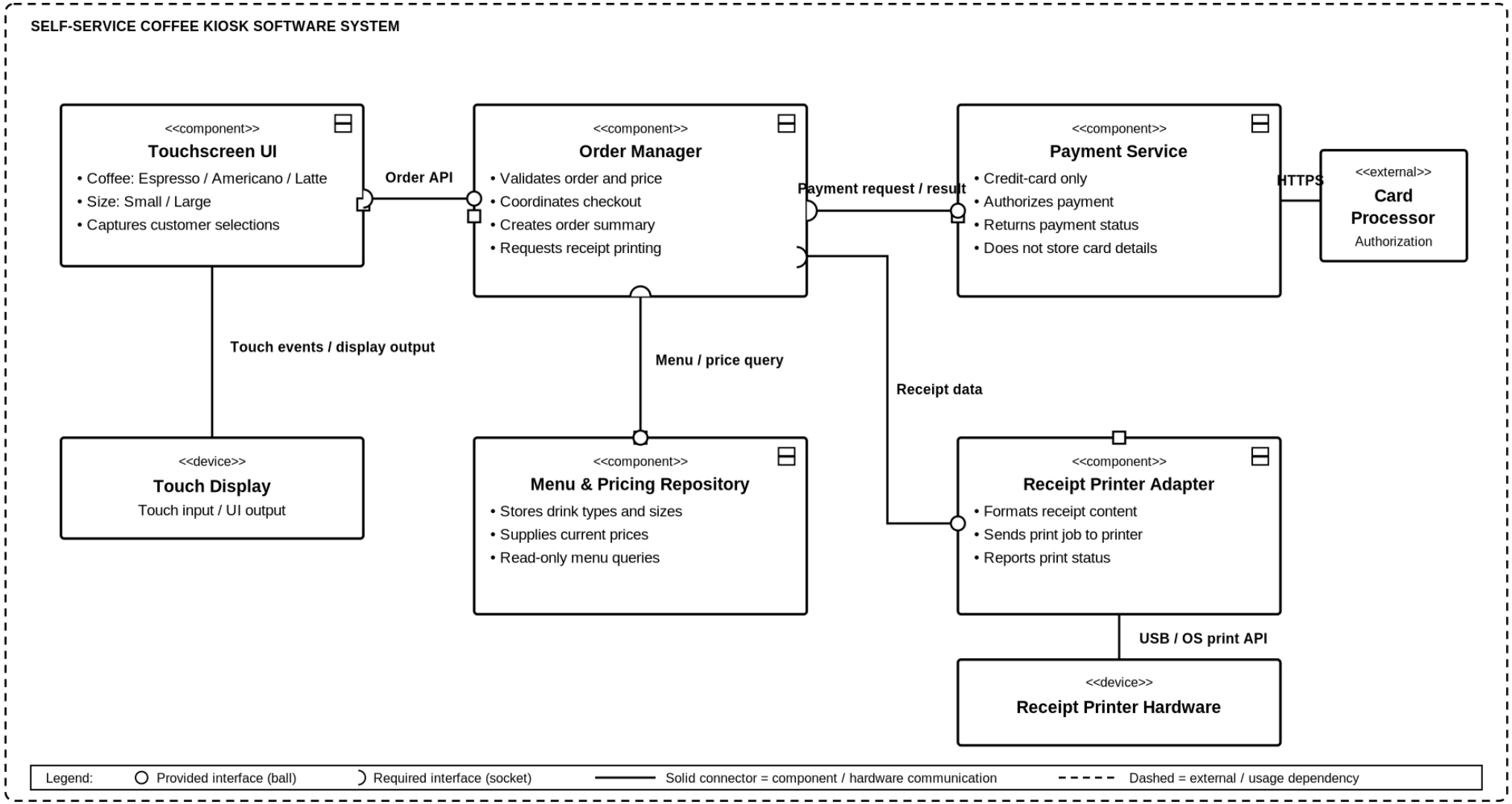
SELF-SERVICE COFFEE KIOSK — UML COMPONENT DIAGRAM

Selected style: Layered Architecture | Credit-card payment | Touchscreen ordering | Receipt printing



Software Engineering – Lab 3

Architecture Selection and Justification

Name – Neha Rastogi

SRN – PES1UG24AM175

System: Self-Service Coffee Kiosk

Architecture: Layered Architecture

Reasons for the choice:

- **Compact, well-defined workflow:** customers choose a drink (Espresso, Americano or Latte) and size (Small or Large), then complete a credit-card payment and receive a printed receipt. Separating presentation, order logic, and data/hardware access keeps each responsibility clear. Second, the system has a small, fixed set of components and one kiosk interaction flow; a layered design is simpler to deploy and maintain than independently deployed microservices, while avoiding unnecessary distributed-service complexity.

- **Security advantage:** The Payment Service is isolated behind a defined interface and handles credit-card authorization without storing card details in the kiosk application. The kiosk can send payment requests to an external payment processor over HTTPS and only retain the authorization result needed to complete the order.

- **Performance benefit:** Menu and price lookups can be served locally from the Menu & Pricing Repository, avoiding unnecessary network calls during drink selection. Keeping the ordering workflow within a single kiosk application also reduces inter-service network latency; only payment authorization depends on the external processor.

- **Components and interactions:** The diagram contains the Touchscreen UI, Order Manager, Payment Service, Menu & Pricing Repository, and Receipt Printer Adapter, plus the touch display, external card processor, and receipt-printer hardware. It labels the order API, payment request/result, menu/price query, receipt data, printer interface, and HTTPS authorization connections.

- **Trade-off:** Layered architecture is straightforward for this kiosk, but it does not provide the independent scaling and fault isolation of microservices. For a single kiosk with a narrow workflow, that trade-off is acceptable.